

Introduction

This report documents the development of two client-based projects completed over the academic year within Unity and Blender. The work focuses on the creation of modular 3D assets, environment design, and gameplay systems, with an emphasis on developing both technical and creative skills through practical game development.

The first project focused on building a low-poly office environment through the creation of reusable assets such as desks, chairs, monitors, and other supporting props. This stage primarily developed my understanding of modelling workflows, optimisation, and how individual assets contribute to a cohesive environment within a game engine.

The second project expanded on these foundations by introducing a more interactive theme park environment. This included not only asset creation but also the development of gameplay systems such as player movement, camera control, build interaction mechanics, and procedural animation. This shift represented a progression from static environment design into functional gameplay development.

Across both projects, I aimed to improve my understanding of industry workflows, including asset pipeline integration between Blender and Unity, iterative development, and problem-solving through testing and experimentation. The work also reflects increasing awareness of scope management, particularly when developing systems independently under time constraints.

Overall, this report presents a year-long development process that demonstrates progression in both technical ability and understanding of game development practices, while also identifying areas for improvement such as documentation, structured testing, and feedback tracking.

Client Project 1 – Office Environment (Asset Creation)

Project Overview

In Client Project 1, the objective was to build an office scene with modular elements using low-poly 3D models made in Blender and implemented in Unity. The primary goal of this

project was to make a set of props that could be used together to create a believable interior scene, keeping in mind real-time performance optimization.

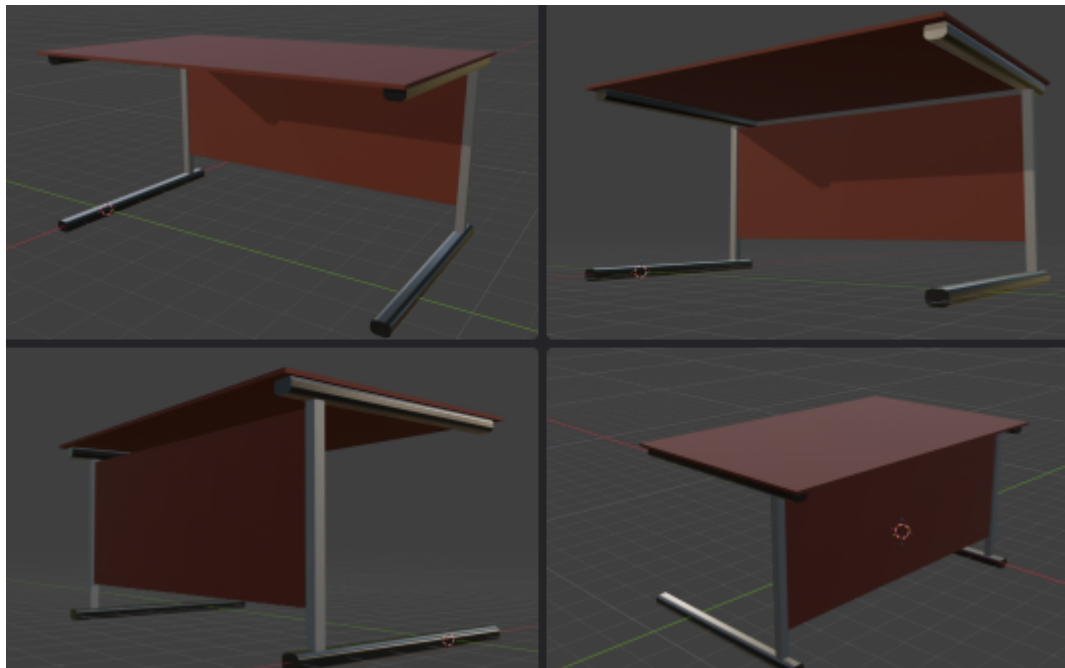
The purpose of this particular project was to get familiar with the process of creating assets, from modelling through materials and their implementation in the engine. In this case, the priority was the creation of assets rather than the development of gameplay features that were explored in Client Project 2.

Desk Asset

This asset was designed with the idea of usage in the office environment and its purpose was to become one of the main structural props in the scene. During the designing process, I tried to design the asset by using simple geometry in Blender in order to make it low-poly and realistic.

While working on the modelling process, I had to test the scale and structure of the asset in order to find out whether it would look natural at a workstation. An important part of the work was the attempt to make the asset not intrusive and at the same time detailed in order to add there other props like monitors, keyboard, etc.

After completing the process of modelling, materials were added to the desk in order to create its professional look. Then I checked in Unity, how the asset looks with the help of different lighting and tried to understand how much influence material and lighting can have on the final look of the asset in the engine.

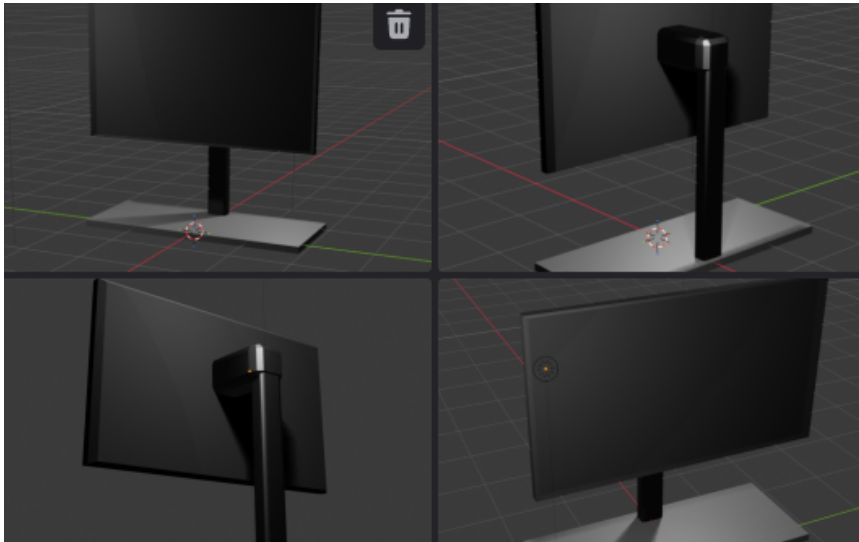


Monitor Asset

The monitor was designed to enhance detail within workstation areas and support the overall office composition. I created the model using cube-based geometry, shaping it into a screen, stand, and base structure.

A key focus during development was avoiding overly flat geometry, so I added thickness to the screen and support components. This helped improve realism while still keeping the model optimised.

Once imported into Unity, I applied darker materials to test how lighting would interact with reflective surfaces. This allowed me to evaluate how well the model fit within the scene and whether any adjustments were needed for visual clarity.



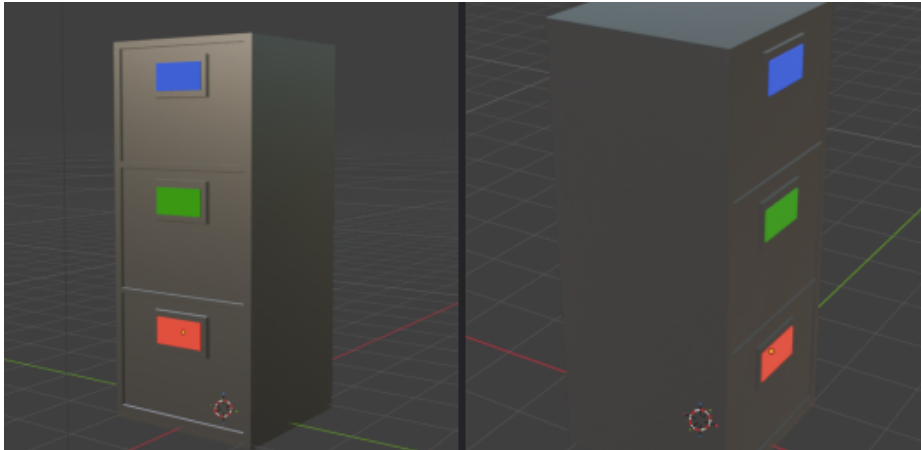
Filing Cabinet Asset

The filing cabinet was created to act as a supporting environmental prop that would help fill empty spaces and improve the realism of the office layout. I used simple shapes in Blender to construct the main structure and divided it into drawer sections to maintain readability.

Colour variation was used on drawer labels to break up the design visually without relying on high-resolution textures. This approach allowed the asset to remain low-poly while still adding visual interest.

The proportions were carefully adjusted to ensure the asset could be reused across multiple areas of the environment without looking out of place. This helped reinforce the idea of

modular design, which was a key focus throughout the project.

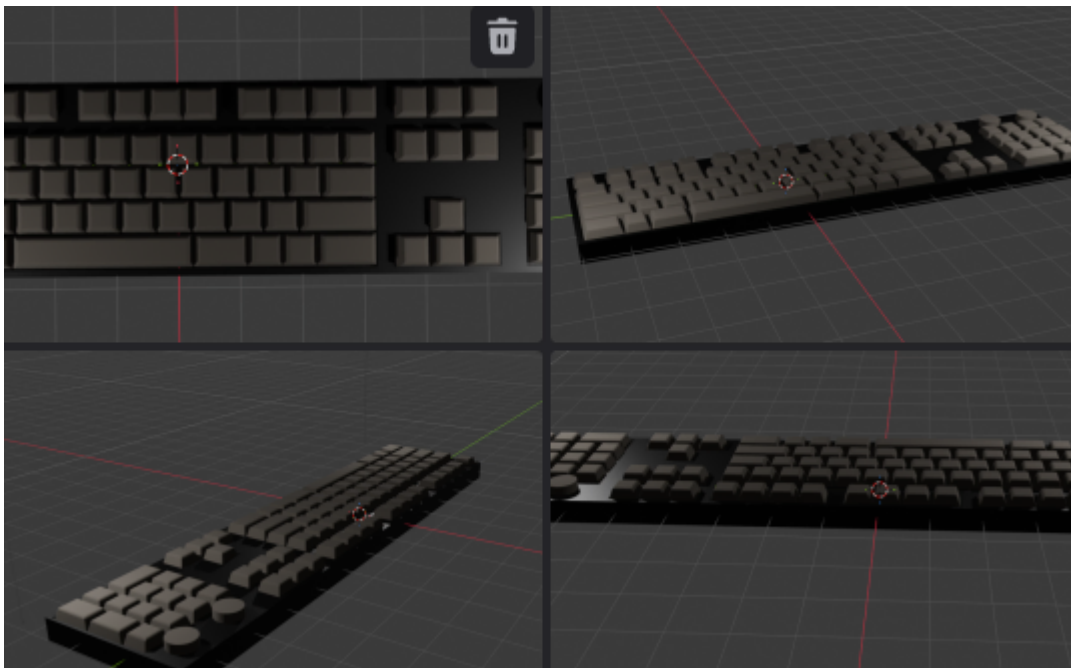


Keyboard Asset

The keyboard was created using a stretched base mesh with repeated key geometry placed across the surface. This allowed me to maintain a structured layout while keeping the model efficient in terms of polygon count.

I added slight height variation to the keys to prevent the model from appearing flat, which improved readability when viewed up close in the engine.

After importing into Unity, I applied dark materials and tested how lighting affected surface visibility. This helped ensure the keyboard remained readable in different lighting conditions while still fitting the overall office style.



Office Chair Asset

The office chair was developed using multiple primitive shapes representing the seat, backrest, armrests, and wheel base. Each component was constructed separately and then combined into a single asset.

The main focus during development was maintaining a balance between realism and optimisation. I avoided unnecessary geometry while still ensuring the silhouette was recognisable as an office chair.

Materials were applied in Unity to differentiate between seating and structural parts, which helped improve visual clarity within the scene.



Cupboard Asset

The cupboard was designed as a modular shelving unit intended to support environmental storytelling and fill empty areas within the office.

I constructed the frame first before dividing the interior into evenly spaced shelves. This helped create a structured and organised appearance, which fits the intended office theme.

Wood-style materials were used on the outer frame to create contrast against lighter internal surfaces. This helped the asset stand out while still fitting the overall visual style of the environment.



Workflow and Engine Testing

All assets were created in Blender and exported into Unity for testing and integration. This workflow allowed me to evaluate:

- Scale accuracy within the environment
- Material and lighting interaction
- Object placement and scene composition
- Overall performance and optimisation

Through repeated testing, I made adjustments to ensure assets were correctly scaled and visually consistent across the scene. This iterative process helped improve my understanding of how Blender and Unity workflows connect in a production pipeline.

Evaluation and Reflection (Client Project 1)

Overall, Client Project 1 helped me develop a stronger understanding of low-poly asset creation and modular environment design. I improved my ability to create reusable assets and gained experience in exporting and testing models within Unity.

However, this stage of development had clear limitations. The work was mostly focused on individual asset creation, and there was limited structured feedback tracking or iteration documentation. At the time, I also did not clearly link development decisions to a broader project narrative, which made it difficult to demonstrate progression over time.

This lack of structured documentation and reflection was something I needed to improve in Client Project 2, where I shifted more towards gameplay systems and interaction-based development.

Client Project 2 – Theme park (Asset Creation and Development)

Client Project 2 built on the foundations of Client Project 1 by expanding from static environment creation into a more interactive and system-driven theme park environment. While the first project focused mainly on asset creation and modular design, this stage introduced gameplay mechanics, interaction systems, camera control, and more complex Unity-based functionality using C#.

The main aim of this project was to develop a more engaging environment that included both decorative assets and functional gameplay elements. This represented a clear progression in my skills, moving from purely visual development into systems-based design and player interaction.

Environment Assets and Development

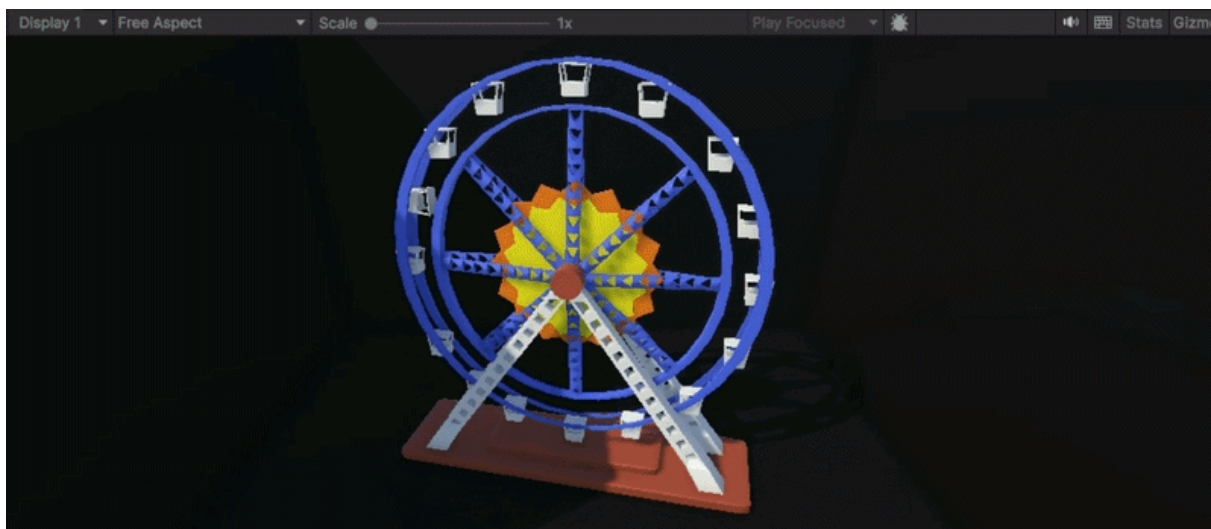
Ferris Wheel Asset

The Ferris wheel was developed in Blender using a modular low-poly approach. I started by constructing the central frame using basic geometric shapes, then built repeating structural components to maintain consistency and efficiency in the modelling process.

Passenger cabins were duplicated around the wheel to ensure uniform spacing and reduce unnecessary modelling time. This approach helped reinforce the importance of modular design, especially for larger-scale environmental objects.

Once imported into Unity, I tested the asset under different lighting conditions to ensure visibility and visual clarity. Materials were applied to distinguish structural elements from cabins, and colour variation was used to improve readability within the theme park environment.

This asset also supported early experimentation with gameplay use, as it was designed to function not only as a visual landmark but also as a potential interactive attraction within the environment.



Trash Can Assets (1 and 2)

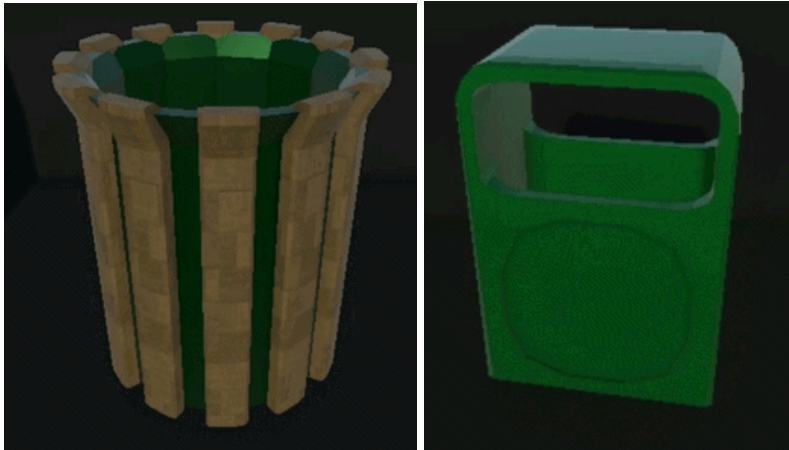
Two variations of trash can assets were created to support environmental storytelling and gameplay mechanics.

The first version was built using a simple low-poly approach in Blender, focusing on recognisable form and fast production. The second version improved on this by refining topology, improving silhouette definition, and adding clearer surface separation.

During development, I experimented with shape adjustments to ensure the object remained identifiable from distance while still fitting the stylised theme park aesthetic.

In Unity, materials and colour values were adjusted to improve visibility and consistency. This also allowed me to test how environmental props could contribute to gameplay systems, as I later linked trash can placement to environmental litter behaviour within the game.

This introduced a basic form of systemic interaction, where player actions influenced environmental conditions.



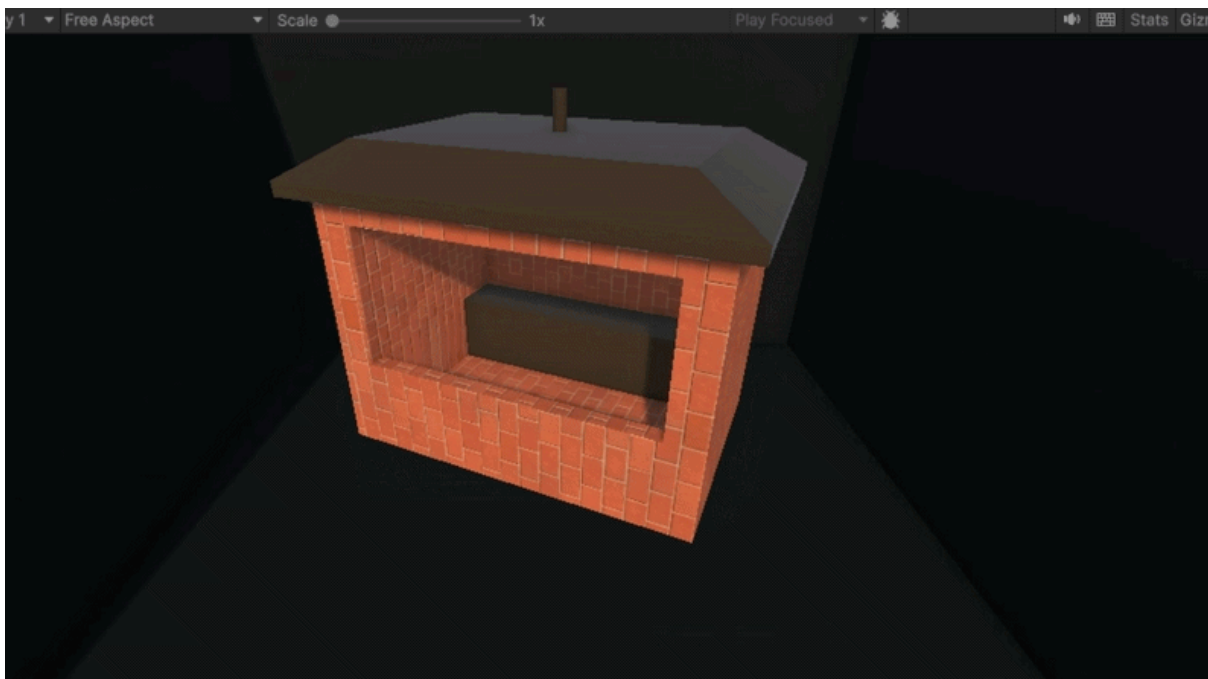
Shop Stall Asset

The shop stall was designed as a modular prop to increase environmental realism and support gameplay interaction within the theme park setting.

The structure was created using cube-based modelling in Blender, with separate components for walls, roof, and serving counter. I extended the roof slightly beyond the front of the stall to improve depth and prevent the model from appearing flat.

In Unity, I applied contrasting materials such as brick textures for walls and darker tones for the roof and counter. This helped ensure the asset stood out visually while still fitting the overall theme.

The stall was also designed with gameplay potential in mind, allowing it to function as an interactive point within the environment.



Circus Tent Asset

The circus tent was developed using a cylindrical base structure combined with an extruded roof to create a recognisable silhouette. Additional geometry was added along the upper edge to introduce subtle detail without increasing the polygon count significantly.

The main focus during development was readability and visual impact from distance, as this asset was intended to act as a key visual landmark within the theme park environment.

In Unity, vibrant red, white, and yellow materials were applied to reinforce the circus theme and increase visual contrast within the environment.

This asset also had potential gameplay applications, as it could be placed as an interactive attraction within the park.



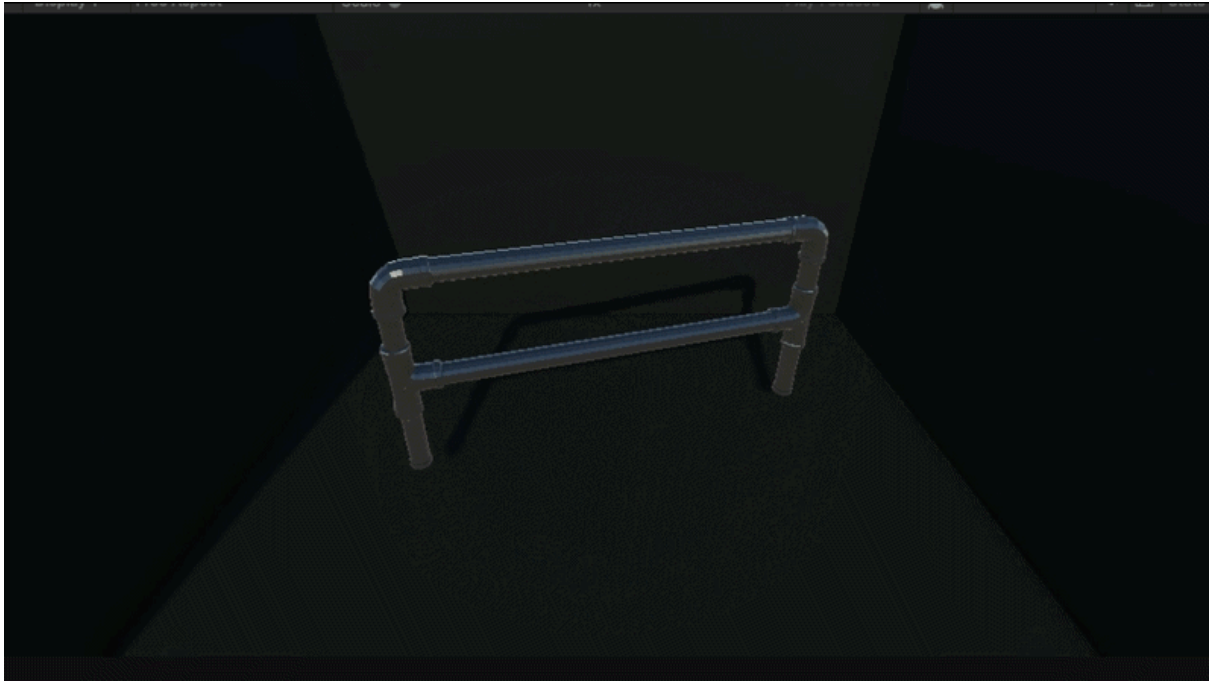
Fence Asset

The fence asset was created using cylindrical and curved pipe-like structures to form a barrier system suitable for guiding player movement within the environment.

I focused on maintaining a clean and readable structure while avoiding unnecessary complexity. Slight curvature was added to reduce rigidity and improve visual realism.

In Unity, metallic materials were applied to give the asset an industrial appearance suitable for a theme park queue or boundary system.

This asset supported both environmental design and gameplay navigation, helping to guide player movement through controlled spaces.

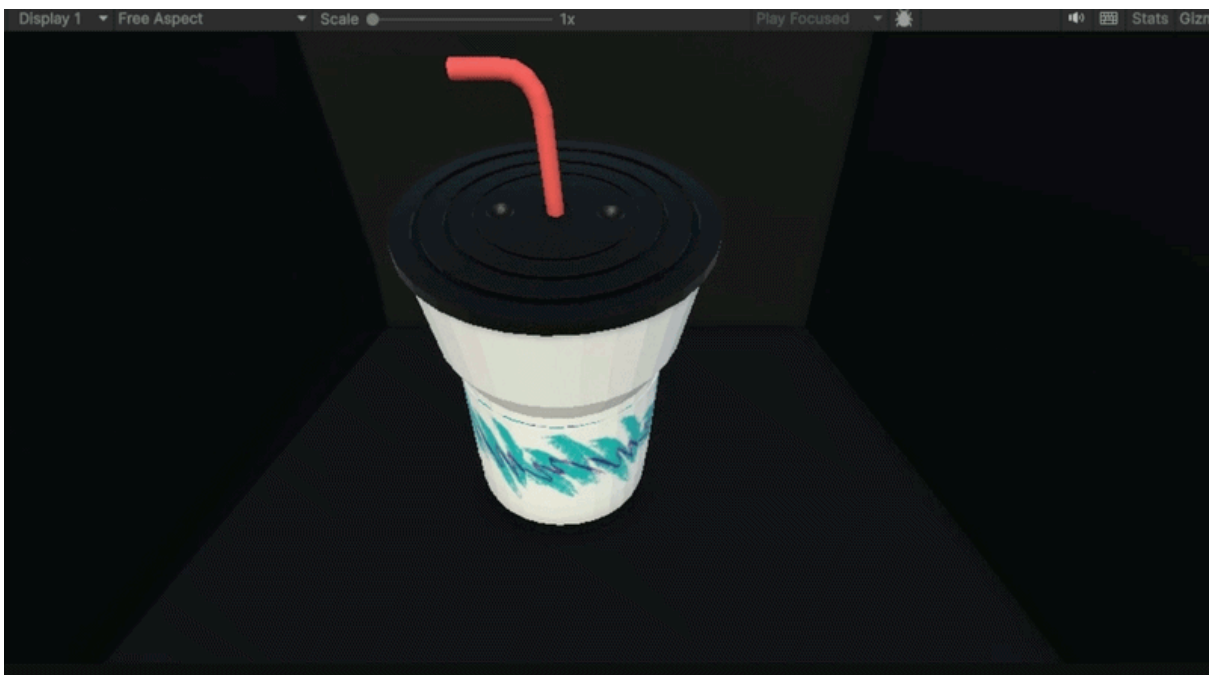


Drink Cup Asset

The drink cup was created using a cylinder-based model with additional geometry for the lid and straw. This allowed the asset to remain simple while still being clearly identifiable.

Different colours were applied to the lid and straw to improve visual clarity and enhance readability when placed in the environment.

In Unity, the asset was positioned near food stalls, benches, and pathways to support environmental storytelling and increase realism within the scene.



Gameplay Systems and Mechanics

Build Camera System

A custom build camera system was implemented in Unity using C#. The purpose of this system was to improve visibility and control during building interactions within the environment.

The camera movement was developed using `Vector3.Lerp()` to create smooth transitions between positions. This helped prevent abrupt movement and made the system feel more polished and responsive.

The camera logic was structured using the `LateUpdate()` function to ensure it updated after player movement, improving stability and reducing visual inconsistencies.

A fixed offset was applied to maintain a consistent viewing angle, and the `LookAt()` function was used to ensure the camera remained focused on the player or target object during build mode.

This system significantly improved usability and demonstrated my understanding of camera control systems used in simulation-style games.

```
1  using UnityEngine;
2  using UnityEngine.Splines;
3
4  public class CameraFollow : MonoBehaviour
5  {
6      public Transform target;
7      public Vector3 offset = new Vector3(0, 5, -8);
8      public float smooth = 5f;
9
10     void LateUpdate()
11     {
12         if (!target) return;
13
14         Vector3 desiredPos = target.position + offset;
15
16         transform.position = Vector3.Lerp(
17             transform.position,
18             desiredPos,
19             Time.deltaTime * smooth
20         );
21
22         transform.LookAt(target);
23     }
24 }
```

Build Plate Interaction System

The build plate system was developed using trigger-based interaction logic in Unity. This system allowed the player to enter specific zones and activate build mode.

The system was structured using:

- `OnTriggerEnter()`

- OnTriggerExit()
- Update()

This separation helped maintain clear logic flow and made the system easier to debug and expand.

The system used player tagging and collision detection to determine when interaction was possible. This ensured build mode only activated under correct conditions, improving reliability and preventing unintended behaviour.

Research into similar interaction systems helped inform the design, particularly in terms of how trigger-based gameplay is used in simulation and building games.

```
1  using UnityEngine;
2
3  public class BuildPlate : MonoBehaviour
4  {
5      bool inside = false;
6
7      void OnTriggerEnter(Collider other)
8      {
9          if (other.CompareTag("Player"))
10             inside = true;
11      }
12
13      void OnTriggerExit(Collider other)
14      {
15          if (other.CompareTag("Player"))
16             inside = false;
17      }
18
19      void Update()
20      {
21          if (inside && Input.GetKeyDown(KeyCode.E))
22          {
23              BuildModeManager.Instance.ToggleBuildMode();
24          }
25      }
26  }
```

Player Controller System

The third-person player controller was created using Unity's CharacterController component. This system allowed movement, jumping, gravity, and camera control to be integrated into a single functional gameplay loop.

The code was structured into modular functions:

- Movement handling
- Gravity handling
- Camera rotation

This approach improved readability and made the system easier to manage and modify.

Research into professional third-person controllers helped influence the design, particularly in relation to smooth movement, grounded detection, and camera constraints.

Several iterations were made to adjust movement speed, jump height, and sensitivity values to improve gameplay feel and responsiveness.

```
void Move()
{
    float x = Input.GetAxis("Horizontal");
    float z = Input.GetAxis("Vertical");

    Vector3 move = transform.right * x + transform.forward * z;
    controller.Move(move * speed * Time.deltaTime);

    // Jump
    if (controller.isGrounded && Input.GetKeyDown(KeyCode.Space))
    {
        yVelocity = Mathf.Sqrt(jumpHeight * -2f * gravity);
    }
}

void HandleGravity()
{
    if (controller.isGrounded && yVelocity < 0)
    {
        yVelocity = -2f; // keeps player grounded
    }

    yVelocity += gravity * Time.deltaTime;
    controller.Move(Vector3.up * yVelocity * Time.deltaTime);
}

void Look()
{
    float mouseX = Input.GetAxis("Mouse X") * mouseSensitivity * Time.deltaTime;
    float mouseY = Input.GetAxis("Mouse Y") * mouseSensitivity * Time.deltaTime;

    xRotation -= mouseY;
    xRotation = Mathf.Clamp(xRotation, -40f, 80f);

    cameraPivot.localRotation = Quaternion.Euler(xRotation, 0f, 0f);
    transform.Rotate(Vector3.up * mouseX);
}
}
```

```
using UnityEngine;

public class ThirdPersonController : MonoBehaviour
{
    public float speed = 6f;
    public float mouseSensitivity = 100f;

    public float jumpHeight = 1.5f;
    public float gravity = -9.81f;

    public Transform cameraPivot;

    float xRotation = 0f;
    float yVelocity;

    CharacterController controller;

    void Start()
    {
        controller = GetComponent<CharacterController>();
        Cursor.lockState = CursorLockMode.Locked;
    }

    void Update()
    {
        Look();
        Move();
        HandleGravity();
    }
}
```

Ferris Wheel Animation System

In addition to the static model, a functional animation system was developed for the Ferris wheel using C#. This system simulated circular motion using sine and cosine functions to position seats evenly around a central axis.

This required an understanding of mathematical motion and transformation systems within Unity.

The system used arrays to manage multiple seat objects and updated their positions dynamically within the Update() function.

This helped improve my understanding of procedural animation and how mathematical functions can be applied to gameplay systems.

```
1 using UnityEngine;
2
3 public class FerrisWheel : MonoBehaviour
4 {
5     public Transform[] seats;
6     public float radius = 100f;
7     public float speed = 20f;
8
9     public Vector3 seatRotationOffset;
10
11     float angle;
12
13     void Update()
14     {
15         angle += speed * Time.deltaTime;
16
17         for (int i = 0; i < seats.Length; i++)
18         {
19             float currentAngle = angle + (360f / seats.Length) * i;
20             float rad = currentAngle * Mathf.Deg2Rad;
21
22             seats[i].position = transform.position + new Vector3(
23                 Mathf.Cos(rad) * radius,
24                 Mathf.Sin(rad) * radius,
25                 0f
26             );
27
28             seats[i].rotation = Quaternion.Euler(seatRotationOffset);
29         }
30     }
31 }
```

Track Builder System (WIP / Incomplete)

The track builder system was an experimental feature designed to allow players to place and extend ride tracks using spline-based logic.

The system involved:

- Raycasting for point placement
- Spline logic for path creation
- Coordinate transformation for alignment
- Build mode state switching

Multiple approaches were tested, including Unity's spline tools, modular piece systems, and runtime mesh generation.

However, several technical issues prevented full completion, including:

- spline rendering instability
- mesh deformation problems
- missing reference errors
- camera and build mode conflicts

Despite multiple iterations and debugging attempts, the system remained incomplete within the project timeframe. However, this experimentation still contributed to my understanding of procedural systems and highlighted the importance of scope management and early technical validation.

Evaluation and Reflection (Client Project 2)

Client Project 2 represents a clear progression from the first project, moving from static asset creation into interactive gameplay systems and functional mechanics.

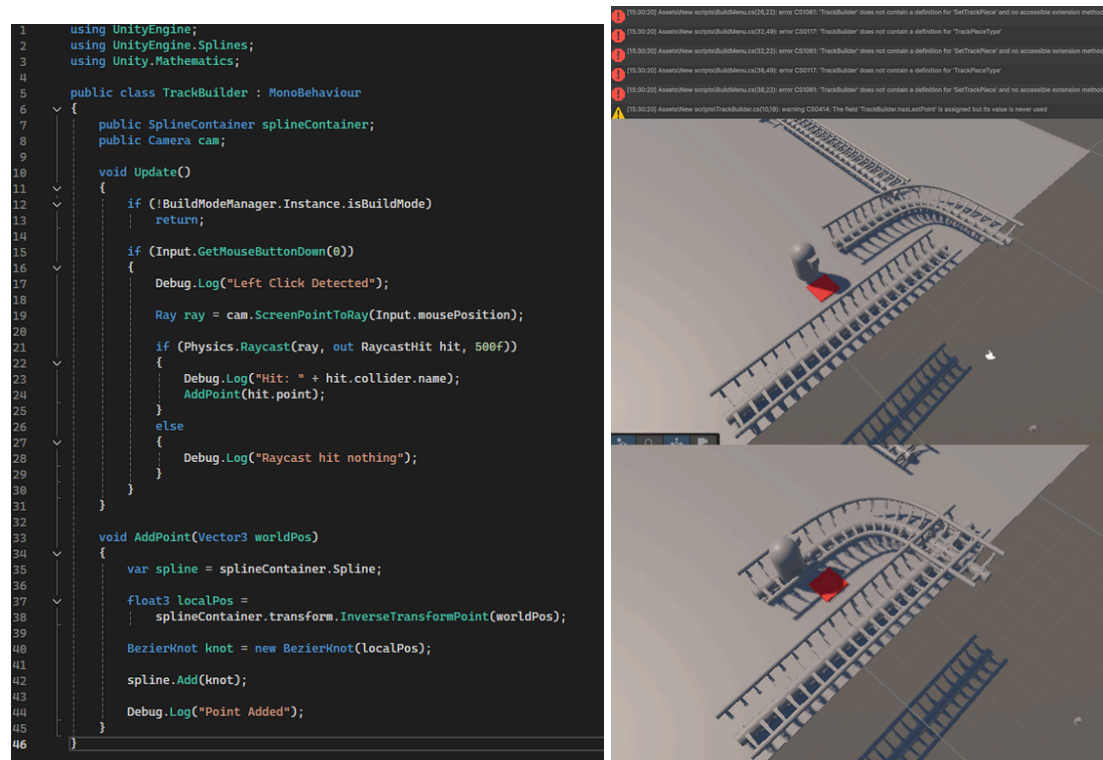
The strongest areas of development included:

- Implementation of player interaction systems
- Camera control and movement logic
- Procedural animation using mathematical functions
- Integration of gameplay mechanics into an environment

This stage also highlighted areas for improvement, particularly in system planning and technical scope control. The unfinished track builder system demonstrates the challenges of working with complex procedural systems without fully stabilised foundations.

Despite this, the project demonstrates increased confidence in Unity development and a stronger understanding of how gameplay systems are structured and implemented in

professional environments.



Client Communication and Project Direction

At the beginning of the project, I discussed my initial ideas with Luke at Frontier Developments to gain feedback on the direction and scope of the work. In this conversation, I presented the overall concept for a game environment system, including early ideas around environment design, gameplay mechanics, and interaction systems.

One of the key pieces of feedback from this discussion was the importance of **scope management**, particularly because I was developing the project individually rather than as part of a team. This meant I needed to ensure that the systems I planned were achievable within the timeframe and did not become overly complex or unmanageable.

As a result of this feedback, I adjusted my approach to focus more on smaller, functional systems rather than attempting to build overly ambitious mechanics early in development. This directly influenced decisions in Client Project 2, where I prioritised working interaction systems such as trigger-based build zones, a basic camera system, and modular gameplay features instead of fully complex systems at an early stage.

Later in development, I revisited communication with the client to discuss progress, challenges, and the direction of the project. At this stage, feedback focused more on reflection and problem-solving rather than initial planning. This helped reinforce the importance of iterative development and adjusting scope based on technical limitations.

However, while communication did occur, the documentation of this feedback process was limited. In future projects, I would improve this by recording structured feedback notes,

including specific comments, responses, and changes made as a result of each discussion. This would allow for clearer evidence of iteration and development over time.

Research, Testing and Iteration Process

Across both projects, research and experimentation played an important role in development, particularly in relation to Unity systems and modelling workflows. In Client Project 1, research was primarily focused on low-poly asset creation techniques, modular design principles, and maintaining optimisation for real-time environments.

In Client Project 2, research expanded into gameplay systems, including:

- Camera control systems used in simulation and building games
- Trigger-based interaction systems in Unity
- Procedural animation using mathematical functions
- Spline and path-based systems for track building mechanics

A key part of development involved testing different approaches in Unity and refining systems based on results. For example, the build camera system required multiple adjustments to smoothing values, offset positioning, and update timing before it behaved consistently.

Similarly, the player controller system went through several iterations to improve movement feel, particularly in relation to jumping and gravity handling. These adjustments were based on testing and observation rather than a single implementation.

However, one of the main weaknesses in the project was the lack of structured documentation of this iteration process. While changes were made during development, I did not always record specific “before and after” states or clearly document how feedback influenced changes. This reduced the visibility of progression within the final report.

The track builder system also highlights this issue clearly. Although multiple approaches were tested (including splines, modular pieces, and raycasting systems), the development process was not fully stabilised early enough, leading to repeated rework and unresolved issues such as mesh instability and spline rendering problems.

This demonstrates that while experimentation was strong, structured iteration tracking needs improvement in future projects.

Knowledge, Understanding and Development Across the Year

Across both Client Project 1 and Client Project 2, there is a clear progression in technical understanding and applied skills.

In Client Project 1, my focus was mainly on:

- Low-poly modelling in Blender
- Modular asset creation
- Unity integration and material testing
- Environmental composition and layout

This phase helped me build foundational skills in asset creation and pipeline workflow, particularly in exporting models and testing them within Unity scenes.

In Client Project 2, this knowledge was expanded significantly into:

- Gameplay programming using C#
- Interaction systems and trigger-based mechanics
- Camera systems and player control
- Procedural animation and mathematical movement systems

This demonstrates a shift from purely visual development into functional gameplay systems, showing clear progression in both technical ability and understanding of game development principles.

However, while technical development improved, the connection between systems and overall player experience was not always clearly documented. In future projects, I would improve this by more explicitly linking design decisions to player experience goals and client expectations.

SDG 8 – Decent Work and Economic Growth

This project can also be linked to SDG 8, which focuses on promoting sustained, inclusive and sustainable economic growth, full and productive employment, and decent work.

Within this context, my project reflects several key professional practices used in the games industry, including:

- Working with real production tools such as Unity and Blender
- Developing modular assets that can be reused efficiently in production pipelines
- Managing scope and workload as an independent developer
- Iterating systems based on testing and technical limitations

The project also reflects real-world constraints commonly seen in game development, such as time management, technical limitations, and feature prioritisation. For example, the decision to reduce the complexity of systems in Client Project 2 reflects industry practice where features are scaled based on feasibility and production time.

Additionally, the development of gameplay systems such as interaction mechanics, camera control, and environment design demonstrates skills relevant to entry-level roles in game development, including environment artist and junior gameplay programmer roles.

However, a limitation in this area is the lack of structured documentation of professional workflows, such as version control (GitHub usage), testing logs, and formal build releases. Improving this would strengthen alignment with industry standards and professional practice expectations.

Overall Reflection and Year-Long Progression

Looking at both projects together, there is a clear development journey from basic environment creation to more complex gameplay systems and interaction design.

Client Project 1 established foundational skills in:

- Asset creation
- Environment design
- Unity integration

Client Project 2 then expanded this into:

- Gameplay systems
- Player interaction mechanics
- Camera control systems
- Procedural and mathematical systems

This shows a clear progression in both technical ability and confidence working within Unity.

However, one of the main weaknesses across the year is the lack of a clearly structured narrative connecting both projects in the original submission. While the work itself demonstrates development, it was not initially presented in a way that clearly showed progression over time.

Another key limitation is the lack of consistent evidence for testing, feedback, and iteration. While these processes did occur during development, they were not fully documented, which made it difficult to demonstrate structured improvement in the final report.

Despite this, the practical work demonstrates engagement with real development processes and shows an understanding of both asset creation and gameplay system design.

Conclusion

Overall, this project has shown a progression in both technical and creative areas over the course of a whole year of production.

Client Project 1 has been based on developing skills in creating low-poly assets and environment design while Client Project 2 has extended this range into game systems development and implementation.

The progress is obvious in the use of Unity and Blender software, as well as understanding of the game systems development, although the major issues within the submission are associated with the lack of documentation, reflection, and iteration.

In order to enhance my work in the future, I should pay more attention to:

- Maintaining the structure of the development during the whole year;
- Documenting the feedback of the clients and iterations;
- Providing the evidence of testing and builds;
- Connecting design decisions with the players' needs and industry trends.

References

sources I used for Inspiration:

thrillville off the rails:

Rollercoaster "Accident" - Jerma Plays Thrillville (Long Edit)

(<https://www.youtube.com/watch?v=7HCLj7m9Z6g>)

Uploaded by dumptruck.

Original footage by jermra985

Thrillville; Off The Rails; Coaster Creator: The Nuke

(<https://www.youtube.com/watch?v=ynPCwzhLgsM>)

Uploaded by BlueThunderKeyKing

Thrillville®: Off the Rails™

(https://store.steampowered.com/app/6080/Thrillville_Off_the_Rails/)

Developed by Frontier developments

Published by LucasArts/LucasFilms game